

Supervised Learning - Model Estimation

100 Questions: OLS, MLE, Gradient Descent, Regularization, Cross-Validation

GARP RAI Exam Preparation

What is Ordinary Least Squares (OLS) estimation in supervised learning?

- ① A method that minimizes the sum of absolute errors between predicted and actual values.
- ② A method that minimizes the sum of squared errors between predicted and actual values.
- ③ A method that maximizes the likelihood of the observed data.
- ④ A method that minimizes the number of parameters in the model.

Answer: B **Explanation:** OLS minimizes $\sum_{i=1}^N (y_i - \hat{y}_i)^2$ - the sum of squared residuals.

Why Others Are Wrong:

- **A:** This describes Least Absolute Deviations (LAD) or L1 loss - a different estimator that minimizes absolute errors.
- **C:** This describes Maximum Likelihood Estimation (MLE) - a different estimation paradigm that maximizes likelihood rather than minimizing errors.
- **D:** This describes regularization (specifically L1/Lasso) which penalizes model complexity - not OLS.

Exam Trap: OLS minimizes **squared errors**, NOT absolute errors! This is the most common confusion on exams.

Which of the following is the correct objective function for Ordinary Least Squares?

- ① $L = \sum_{i=1}^N |y_i - \hat{y}_i|$
- ② $L = \sum_{i=1}^N (y_i - \hat{y}_i)^2$
- ③ $L = \sum_{i=1}^N (y_i - \hat{y}_i)^4$
- ④ $L = \max_i |y_i - \hat{y}_i|$

Answer: B **Explanation:** OLS minimizes the Residual Sum of Squares (RSS): $\sum (y_i - \hat{y}_i)^2$.

Why Others Are Wrong:

- **A:** This is L1 loss (Least Absolute Deviations) - minimizes absolute errors, not squared errors.
- **C:** Higher-order loss (L4) - rarely used in practice; doesn't correspond to OLS.
- **D:** Chebyshev loss (minimax) - minimizes maximum absolute error, not sum of squared errors.

Exam Trap: OLS = **RSS** = $\sum (y_i - \hat{y}_i)^2$! Don't confuse with MAE or other loss functions. The square term is the key identifier.

What is the key difference between Ordinary Least Squares (OLS) and Nonlinear Least Squares (NLS)?

- ① OLS minimizes absolute errors while NLS minimizes squared errors.
- ② OLS is used when the model is linear in parameters; NLS is used when the model is nonlinear in parameters.
- ③ OLS requires gradient descent while NLS has a closed-form solution.
- ④ OLS is for classification while NLS is for regression.

Answer: B Explanation: OLS requires linearity in parameters (e.g., $y = \beta_0 + \beta_1 x$). NLS handles nonlinearity in parameters (e.g., $y = \beta_0 + \beta_1 e^{\beta_2 x}$).

Why Others Are Wrong:

- **A:** Both OLS and NLS minimize squared errors - this is a fundamental misunderstanding of both methods.
- **C:** Actually reversed! OLS has closed-form solution $\hat{\beta} = (X^T X)^{-1} X^T y$; NLS requires iterative methods.
- **D:** Both are regression methods (continuous dependent variables), not classification.

Exam Trap: The difference is **linearity in parameters**, NOT in variables! $y = \beta_0 + \beta_1 x^2$ is OLS because it's linear in parameters, even though it's nonlinear in x .

Q4: Nonlinear Least Squares - Example

Which of the following models would require Nonlinear Least Squares (NLS) estimation?

- ① $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \varepsilon$
- ② $y = \beta_0 + \beta_1 x + \beta_2 x^2 + \varepsilon$
- ③ $y = \beta_0 + \beta_1 e^{\beta_2 x} + \varepsilon$
- ④ $y = \beta_0 + \beta_1 \ln(x) + \varepsilon$

Answer: C **Explanation:** Option C has $\beta_1 e^{\beta_2 x}$ which is nonlinear in parameter β_2 .

Why Others Are Wrong:

- **A:** This is a standard linear regression - linear in parameters $\beta_0, \beta_1, \beta_2$.
- **B:** Polynomial regression - linear in parameters despite being nonlinear in x .
- **D:** Log transformation - linear in parameters (treat $\ln(x)$ as transformed feature).

Exam Trap: NLS is about **nonlinearity in parameters**, not variables! This is the single most common mistake on this topic.

What is Maximum Likelihood Estimation (MLE)?

- ❶ A method that minimizes the sum of squared errors.
- ❷ A method that finds the parameter values that maximize the probability (likelihood) of observing the given data.
- ❸ A method that minimizes the number of parameters in the model.
- ❹ A method that finds the parameter values that minimize the variance of the errors.

Answer: B Explanation: MLE finds θ that maximizes $L(\theta) = P(\text{data}|\theta)$. It asks: "What parameters make the observed data most probable?"

Why Others Are Wrong:

- ❶ **A:** This describes OLS (sum of squared errors) - a completely different estimation paradigm.
- ❸ **C:** This describes regularization (penalizing complexity) - not MLE.
- ❹ **D:** Minimizing error variance is related to efficiency but is not the definition of MLE.

Exam Trap: MLE maximizes **likelihood**, NOT minimizes error! This is the fundamental difference from OLS. Also, likelihood = $P(\text{data}|\theta)$, NOT $P(\theta|\text{data})$ (that's Bayesian posterior).

Under what condition do OLS and MLE produce the same parameter estimates for a linear regression model?

- ❶ When the errors are independent.
- ❷ When the errors are normally distributed with constant variance.
- ❸ When the errors are uniformly distributed.
- ❹ When the errors are autocorrelated.

Answer: B Explanation: When $\varepsilon \sim N(0, \sigma^2)$, log-likelihood is proportional to negative RSS. Maximizing likelihood = minimizing RSS.

Why Others Are Wrong:

- ❶ **A:** Independence alone is insufficient - we need the specific normal distribution form.
- ❸ **C:** Uniform errors would give a different likelihood function (MLE would minimize maximum absolute error).
- ❹ **D:** Autocorrelation violates OLS assumptions and would not lead to OLS-MLE equivalence.

Exam Trap: OLS and MLE are **equivalent** under normal errors! This is a critical exam concept - not just independence, but specifically normal distribution.

What estimation method is typically used for logistic regression?

- ① Ordinary Least Squares (OLS)
- ② Nonlinear Least Squares (NLS)
- ③ Maximum Likelihood Estimation (MLE)
- ④ Method of Moments

Answer: C Explanation: Logistic regression uses MLE with Bernoulli likelihood. The log-likelihood is $\sum [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$.

Why Others Are Wrong:

- **A:** OLS inappropriate because target is binary (0/1) and errors are not normal. OLS would produce heteroscedastic errors and predictions outside $[0,1]$.
- **B:** NLS is not the standard approach; logistic regression's nonlinearity is in the link function, handled naturally by MLE.
- **D:** Method of Moments is less common and less efficient for logistic regression.

Exam Trap: Logistic regression uses **MLE**, NOT OLS! This is a critical distinction for classification problems.

What does the likelihood function $L(\theta|\text{data})$ represent?

- 1 The probability of the parameters given the data.
- 2 The probability of the data given the parameters.
- 3 The sum of squared errors for a given parameter set.
- 4 The variance of the parameter estimates.

Answer: B Explanation: $L(\theta|\text{data}) = P(\text{data}|\theta)$. It's the probability of observing the data given the parameters.

Why Others Are Wrong:

- A: This describes the posterior distribution in Bayesian inference ($P(\theta|\text{data})$).
- C: This is the OLS loss function (RSS) - a different concept.
- D: This describes the variance of estimators - a different statistical concept.

Exam Trap: Likelihood = $P(\text{data}|\theta)$, NOT $P(\theta|\text{data})$! This confusion between likelihood and posterior is extremely common on exams.

Why is the log-likelihood often used instead of the likelihood function?

- ① Because the log-likelihood is always positive.
- ② Because the log-likelihood is easier to maximize (it turns products into sums and is monotonic).
- ③ Because the log-likelihood has a closed-form solution.
- ④ Because the log-likelihood minimizes the number of parameters.

Answer: B Explanation: Log transforms products to sums (easier to differentiate). Since log is monotonic, maximizing log-likelihood $\equiv$ maximizing likelihood.

Why Others Are Wrong:

- **A:** False - log-likelihood is often negative (since probabilities are < 1 , logs are negative).
- **C:** False - log-likelihood usually requires numerical optimization (except simple cases).
- **D:** False - log-likelihood doesn't affect parameter count; that's regularization.

Exam Trap: Log-likelihood is **monotonic** - maximizing it is equivalent to maximizing likelihood! This is why we use it despite often being negative.

Which estimation method typically has a closed-form (analytical) solution?

- ❶ Maximum Likelihood Estimation for logistic regression
- ❷ Nonlinear Least Squares
- ❸ Ordinary Least Squares for linear regression
- ❹ Maximum Likelihood Estimation for neural networks

Answer: C **Explanation:** OLS for linear regression has closed-form: $\hat{\beta} = (X^T X)^{-1} X^T y$.

Why Others Are Wrong:

- ❶ **A:** Logistic regression MLE requires numerical optimization (Newton-Raphson, IRLS).
- ❷ **B:** NLS requires iterative numerical optimization (no closed-form).
- ❹ **D:** Neural networks use backpropagation (numerical gradient-based optimization).

Exam Trap: Only **OLS for linear regression** has a closed-form solution! All other methods require numerical optimization. This is a key distinguishing feature.

Which of the following is NOT a standard assumption of Ordinary Least Squares?

- ① Linearity in parameters
- ② Homoscedasticity (constant variance of errors)
- ③ Normality of errors
- ④ The errors are autocorrelated

Answer: D Explanation: OLS assumes: linearity in parameters, homoscedasticity, independence (no autocorrelation), and (for inference) normality.

Why Others Are Wrong:

- **A:** This IS an OLS assumption - model must be linear in parameters.
- **B:** This IS an OLS assumption - constant error variance (homoscedasticity).
- **C:** This IS an OLS assumption (for hypothesis testing and confidence intervals).

Exam Trap: OLS assumes **no autocorrelation** in errors! Autocorrelation violates the independence assumption. Option D is the violation, not the assumption.

Which of the following is a desirable property of Maximum Likelihood Estimators?

- ❶ They are always unbiased.
- ❷ They are consistent (converge to true values as sample size increases).
- ❸ They always have the smallest variance among all estimators.
- ❹ They never require numerical optimization.

Answer: B **Explanation:** MLE is consistent: $\hat{\theta}_{MLE} \xrightarrow{P} \theta_0$ as $n \rightarrow \infty$ (under regularity conditions).

Why Others Are Wrong:

- ❶ **A:** False - MLE can be biased (e.g., variance estimator $\hat{\sigma}_{MLE}^2$ is biased).
- ❸ **C:** False - MLE is asymptotically efficient but not always minimum variance in finite samples.
- ❹ **D:** False - MLE often requires numerical optimization (logistic regression, etc.).

Exam Trap: MLE is **consistent** but NOT necessarily unbiased! This is a common misconception - consistency and unbiasedness are different properties.

Why does Nonlinear Least Squares typically require iterative numerical optimization?

- ❶ Because the objective function has multiple local minima.
- ❷ Because there is no closed-form solution for nonlinear-in-parameters models.
- ❸ Because the errors are not normally distributed.
- ❹ Because the model has too many parameters.

Answer: B Explanation: NLS requires numerical optimization because the normal equations are nonlinear in parameters and have no closed-form solution.

Why Others Are Wrong:

- ❶ **A:** Multiple local minima are a potential issue but not the primary reason for numerical methods.
- ❸ **C:** Error distribution doesn't determine the need for numerical optimization.
- ❹ **D:** Parameter count affects complexity but isn't the fundamental reason.

Exam Trap: No closed-form solution exists for **nonlinear-in-parameters** models! This is the fundamental reason for iterative methods, not local minima or parameter count.

What does the Gauss-Markov theorem state about OLS estimators?

- ① OLS estimators are always unbiased.
- ② OLS estimators are the Best Linear Unbiased Estimators (BLUE) under certain conditions.
- ③ OLS estimators always have the smallest variance among all estimators.
- ④ OLS estimators are consistent for any distribution of errors.

Answer: B Explanation: Under linearity, no perfect multicollinearity, zero conditional mean, homoscedasticity, OLS is BLUE.

Why Others Are Wrong:

- **A:** Close but incomplete - misses the "Best" (minimum variance) part.
- **C:** False - only among linear unbiased estimators, not all estimators.
- **D:** False - consistency requires additional conditions beyond Gauss-Markov.

Exam Trap: OLS is **BLUE** under Gauss-Markov assumptions! Not "always best" - only among linear unbiased estimators. Non-linear estimators could have lower variance.

Which estimation method would you use for a model where $y = \beta_0 + \beta_1 x_1^{\beta_2} + \varepsilon$?

- ① Ordinary Least Squares (OLS)
- ② Nonlinear Least Squares (NLS)
- ③ Maximum Likelihood Estimation (MLE)
- ④ Both NLS and MLE would be appropriate

Answer: D Explanation: Model is nonlinear in parameter β_2 . Both NLS (minimizes squared errors) and MLE (maximizes likelihood) can be used.

Why Others Are Wrong:

- **A:** OLS requires linearity in parameters - this model violates that.
- **B:** NLS alone is correct but incomplete - MLE also works.
- **C:** MLE alone is correct but incomplete - NLS also works.

Exam Trap: For nonlinear models, both **NLS and MLE** can be used! Choice depends on error distribution assumptions. If errors are normal, they're equivalent.

What is gradient descent in the context of model estimation?

- ❶ A method that finds the global minimum of any function.
- ❷ An iterative optimization algorithm that updates parameters in the direction of the negative gradient of the loss function.
- ❸ A method that solves for parameters using a closed-form equation.
- ❹ A method that randomly searches for optimal parameters.

Answer: B Explanation: Update rule: $\theta_{new} = \theta_{old} - \alpha \nabla L(\theta)$. It's iterative and uses the negative gradient direction.

Why Others Are Wrong:

- ❶ **A:** False - gradient descent can get stuck in local minima; no guarantee of global minimum.
- ❸ **C:** This describes closed-form solutions (like OLS), not gradient descent.
- ❹ **D:** This describes random search or Monte Carlo methods, not gradient descent.

Exam Trap: Gradient descent is **iterative** and uses the **negative gradient**! Not a direct solution method, and doesn't guarantee global minimum.

What is the general update rule for gradient descent?

- ❶ $\theta_{new} = \theta_{old} + \alpha \nabla L(\theta)$
- ❷ $\theta_{new} = \theta_{old} - \alpha \nabla L(\theta)$
- ❸ $\theta_{new} = \theta_{old} \times \alpha \nabla L(\theta)$
- ❹ $\theta_{new} = \theta_{old} / \alpha \nabla L(\theta)$

Answer: B Explanation: $\theta_{new} = \theta_{old} - \alpha \nabla L(\theta)$. We subtract the gradient to move in the direction of decreasing loss.

Why Others Are Wrong:

- ❶ **A:** This is gradient ascent (moving to increase loss) - would maximize rather than minimize.
- ❸ **C:** Multiplication is not the correct operation; gradient descent uses subtraction.
- ❹ **D:** Division is not the correct operation; this is not a valid update rule.

Exam Trap: Gradient descent uses **subtraction** of the gradient! Addition would be gradient ascent (for maximization).

What does the learning rate α control in gradient descent?

- ❶ The direction of the parameter update.
- ❷ The step size of each parameter update.
- ❸ The number of iterations.
- ❹ The initial parameter values.

Answer: B Explanation: α controls step size. Larger α = larger steps (faster but risk overshooting). Smaller α = smaller steps (slower but more stable).

Why Others Are Wrong:

- ❶ **A:** Direction is determined by the gradient sign, not the learning rate.
- ❸ **C:** Number of iterations is a separate hyperparameter (epochs).
- ❹ **D:** Initial parameter values are set separately before optimization.

Exam Trap: Learning rate controls **step size**, not direction! Direction comes from the gradient. This distinction is crucial for understanding optimization.

Q19: Learning Rate - Too Large

What happens if the learning rate is set too large in gradient descent?

- ❶ The algorithm converges very slowly.
- ❷ The algorithm may oscillate or diverge, failing to converge.
- ❸ The algorithm always finds the global minimum.
- ❹ The algorithm becomes deterministic.

Answer: B Explanation: Too large α causes overshooting, oscillation, or divergence. The updates become too large and unstable; loss may increase without bound.

Why Others Are Wrong:

- ❶ **A:** This describes a learning rate that is too small (slow convergence).
- ❸ **C:** Too large learning rate prevents convergence; it certainly doesn't guarantee global minimum.
- ❹ **D:** The algorithm is already deterministic (given fixed initialization and data).

Exam Trap: Too large learning rate $\rightarrow$ **divergence or oscillation!** This is a common failure mode. The opposite (too small) gives slow convergence.

What happens if the learning rate is set too small in gradient descent?

- ❶ The algorithm converges very quickly.
- ❷ The algorithm converges very slowly and may get stuck.
- ❸ The algorithm always finds the global minimum instantly.
- ❹ The algorithm becomes unstable.

Answer: B Explanation: Too small α = tiny steps, very slow convergence. May get stuck in local minima or plateaus because updates are too small to escape.

Why Others Are Wrong:

- ❶ **A:** False - too small gives slow convergence, not quick.
- ❸ **C:** False - no algorithm finds global minimum instantly.
- ❹ **D:** Too small is stable but slow; instability comes from too large learning rate.

Exam Trap: Too small learning rate $\rightarrow$ **slow convergence and getting stuck!** Need to balance between too large (divergence) and too small (slow).

What is Batch Gradient Descent?

- ❶ Gradient descent that updates parameters using a single randomly selected data point.
- ❷ Gradient descent that updates parameters using the entire training dataset.
- ❸ Gradient descent that updates parameters using a small random subset of data.
- ❹ Gradient descent that updates parameters without using any data.

Answer: B Explanation: Batch GD computes gradient using the entire dataset for each parameter update. Computationally expensive for large datasets.

Why Others Are Wrong:

- **A:** This describes Stochastic Gradient Descent (SGD) - uses one data point.
- **C:** This describes Mini-Batch Gradient Descent - uses a subset.
- **D:** This describes random search - not gradient descent.

Exam Trap: Batch GD uses **ALL data** for each update! This is the key differentiator from SGD and mini-batch.

What is Stochastic Gradient Descent (SGD)?

- ❶ Gradient descent that updates parameters using the entire dataset.
- ❷ Gradient descent that updates parameters using a single randomly selected data point per iteration.
- ❸ Gradient descent that updates parameters using a small random subset of data.
- ❹ Gradient descent that uses a fixed learning rate schedule.

Answer: B Explanation: SGD updates parameters using one randomly selected data point per iteration. Fast and helps escape local minima due to gradient noise.

Why Others Are Wrong:

- **A:** This describes Batch GD (uses all data).
- **C:** This describes Mini-Batch GD (uses subset).
- **D:** This describes learning rate scheduling - a separate concept.

Exam Trap: SGD uses **ONE data point** per update! This is the key differentiator - not batch, not mini-batch.

What is Mini-Batch Gradient Descent?

- ❶ Gradient descent that updates parameters using the entire dataset.
- ❷ Gradient descent that updates parameters using a single data point.
- ❸ Gradient descent that updates parameters using a small random subset (batch) of data.
- ❹ Gradient descent that uses a fixed number of iterations.

Answer: C Explanation: Mini-Batch GD balances Batch GD (stable) and SGD (fast). Uses small batch (e.g., 32, 64, 128 samples) per update.

Why Others Are Wrong:

- ❶ **A:** This is Batch GD (all data).
- ❷ **B:** This is SGD (one data point).
- ❹ **D:** This describes a stopping criterion, not the gradient method.

Exam Trap: Mini-Batch GD uses a **small random subset** of data! This is the most common in practice - balances stability and speed.

Why is Stochastic Gradient Descent (SGD) often preferred over Batch Gradient Descent for large datasets?

- ❶ SGD always converges faster than Batch GD.
- ❷ SGD is computationally cheaper per iteration because it uses fewer data points.
- ❸ SGD always finds the global minimum.
- ❹ SGD does not require a loss function.

Answer: B Explanation: SGD cheaper per iteration because uses fewer data points (one or small batch) to compute gradient. Scalable to large datasets.

Why Others Are Wrong:

- **A:** Not always - SGD converges noisily and may require more iterations.
- **C:** False - SGD doesn't guarantee global minimum (no method does).
- **D:** False - SGD requires a loss function to compute gradients.

Exam Trap: SGD preferred for large datasets due to **lower computational cost per iteration**, not because it always converges faster.

What is a local minimum in gradient descent?

- ❶ The point where the loss function is at its global minimum.
- ❷ The point where the gradient is zero but the function value is higher than at other nearby points.
- ❸ The point where the gradient is infinite.
- ❹ The point where the loss function is at its maximum.

Answer: B Explanation: Local minimum: gradient ≈ 0 , function value lower than neighbors, but not globally lowest. Gradient descent can get stuck here.

Why Others Are Wrong:

- ❶ **A:** This describes global minimum (lowest point overall).
- ❸ **C:** Gradient is zero at minima, not infinite.
- ❹ **D:** This describes a maximum (gradient zero, higher than neighbors).

Exam Trap: Local minima are **traps** where gradient descent can get stuck! They have zero gradient but aren't the global best.

What is a saddle point in gradient descent?

- ❶ A point where the gradient is zero and the function curves upward in all directions.
- ❷ A point where the gradient is zero but the function curves upward in some directions and downward in others.
- ❸ A point where the gradient is infinite.
- ❹ A point where the function is at its global minimum.

Answer: B Explanation: Saddle point: gradient zero, neither local min nor max. Curves upward in some directions, downward in others. Common in high dimensions.

Why Others Are Wrong:

- ❶ **A:** This describes a local minimum (curves upward in all directions).
- ❸ **C:** Gradient is zero at saddle points, not infinite.
- ❹ **D:** This describes global minimum - saddle points are not minima.

Exam Trap: Saddle points have **zero gradient** but are neither minima nor maxima! More common than local minima in high dimensions.

What is the purpose of momentum in gradient descent?

- ❶ To decrease the learning rate over time.
- ❷ To accelerate convergence and reduce oscillations by accumulating past gradients.
- ❸ To increase the batch size.
- ❹ To randomly initialize parameters.

Answer: B Explanation: Momentum adds fraction of previous update: $v_t = \beta v_{t-1} + (1 - \beta) \nabla L(\theta)$, $\theta = \theta - \alpha v_t$. Smooths oscillations, helps escape shallow minima.

Why Others Are Wrong:

- ❶ **A:** This describes learning rate decay - a separate concept.
- ❸ **C:** This describes batch size - unrelated to momentum.
- ❹ **D:** This describes parameter initialization - a different step.

Exam Trap: Momentum uses **past gradients** to smooth updates! This helps accelerate convergence and escape local minima.

What is the Adam optimizer in gradient descent?

- ❶ A method that uses a fixed learning rate.
- ❷ An adaptive learning rate optimizer that combines momentum and RMSprop.
- ❸ A method that only uses the first derivative.
- ❹ A method that randomly initializes parameters.

Answer: B Explanation: Adam = Adaptive Moment Estimation. Combines momentum (first moment) and RMSprop (second moment). Adapts learning rate per parameter.

Why Others Are Wrong:

- ❶ **A:** Adam adapts learning rate dynamically; it's not fixed.
- ❸ **C:** Adam uses first AND second moments - more than just first derivative.
- ❹ **D:** Adam is an optimizer, not an initialization method.

Exam Trap: Adam is **adaptive** - adjusts learning rates per parameter! Most popular optimizer in deep learning.

What is learning rate scheduling?

- ❶ A method that randomly changes the learning rate.
- ❷ A method that adjusts the learning rate during training according to a predefined schedule.
- ❸ A method that keeps the learning rate constant throughout training.
- ❹ A method that changes the batch size during training.

Answer: B Explanation: Learning rate scheduling adjusts LR during training (step decay, exponential decay, cyclical). Larger steps early, smaller steps later.

Why Others Are Wrong:

- **A:** Random changes would be unpredictable and not a "schedule".
- **C:** This is a constant learning rate - the opposite of scheduling.
- **D:** This describes batch size scheduling - a different concept.

Exam Trap: Learning rate schedules **adjust the learning rate over time!** Not random, not constant, not batch size.

What is step decay in learning rate scheduling?

- ❶ The learning rate decreases continuously over time.
- ❷ The learning rate decreases by a factor at specific intervals (e.g., every 10 epochs).
- ❸ The learning rate increases over time.
- ❹ The learning rate is randomly changed.

Answer: B Explanation: Step decay reduces LR by factor (e.g., multiply by 0.1) at specific intervals (e.g., every 10 epochs or after certain iterations).

Why Others Are Wrong:

- ❶ **A:** This describes exponential decay (continuous decrease).
- ❸ **C:** This describes increasing learning rate (rarely used).
- ❹ **D:** This describes random changes - not a defined schedule.

Exam Trap: Step decay reduces learning rate **at specific intervals!** This is a key distinction from exponential decay (continuous).

What is exponential decay in learning rate scheduling?

- ❶ $\alpha_t = \alpha_0 \times \gamma^t$
- ❷ $\alpha_t = \alpha_0 / t$
- ❸ $\alpha_t = \alpha_0 \times (1 - \gamma t)$
- ❹ $\alpha_t = \alpha_0$ (constant)

Answer: A **Explanation:** Exponential decay: $\alpha_t = \alpha_0 \times \gamma^t$ where γ is decay rate (typically 0.9-0.99). Smooth continuous decay.

Why Others Are Wrong:

- ❷ **B:** This is inverse time decay ($\alpha_t = \alpha_0 / t$) - different schedule.
- ❸ **C:** This is linear decay - decreases linearly, not exponentially.
- ❹ **D:** This is constant learning rate - no decay at all.

Exam Trap: Exponential decay: $\alpha_t = \alpha_0 \times \gamma^t$! Know the formula and how it differs from step decay and inverse decay.

What does convergence mean in gradient descent?

- ❶ The algorithm has found the global minimum.
- ❷ The algorithm has stopped updating parameters.
- ❸ The loss function has stopped decreasing significantly (or the gradient is near zero).
- ❹ The algorithm has reached the maximum number of iterations.

Answer: C Explanation: Convergence = loss stopped decreasing significantly or gradient near zero. Found a minimum (local or global).

Why Others Are Wrong:

- ❶ **A:** Ideal but not guaranteed - could be local minimum.
- ❷ **B:** Could be due to learning rate = 0, not convergence.
- ❹ **D:** This is a stopping criterion, not convergence.

Exam Trap: Convergence means **loss has stopped decreasing** significantly! Not necessarily global minimum, not just stopping updates.

In high-dimensional spaces (e.g., neural networks), which is more common: local minima or saddle points?

- 1 Local minima are more common.
- 2 Saddle points are more common.
- 3 Global minima are always found.
- 4 Neither local minima nor saddle points exist.

Answer: B Explanation: In high dimensions, saddle points are more common than local minima. Research shows local minima in neural network landscapes are relatively rare.

Why Others Are Wrong:

- **A:** False - local minima are rare in high dimensions.
- **C:** False - no guarantee of finding global minimum.
- **D:** False - both exist, but saddle points are more common.

Exam Trap: Saddle points are **more common** than local minima in high dimensions! This is a key insight for deep learning optimization.

What is the vanishing gradient problem in deep neural networks?

- ❶ Gradients become very large during backpropagation.
- ❷ Gradients become very small during backpropagation, preventing early layers from learning.
- ❸ Gradients oscillate wildly during training.
- ❹ Gradients are zero for all parameters.

Answer: B **Explanation:** Vanishing gradients: gradients become very small during backpropagation, especially in early layers. Prevents effective updates. Often caused by sigmoid/tanh in saturation regions.

Why Others Are Wrong:

- ❶ **A:** This describes exploding gradients (large gradients).
- ❸ **C:** This describes oscillations - a different problem.
- ❹ **D:** Extreme case - gradients becoming exactly zero everywhere.

Exam Trap: Vanishing gradients = **very small gradients** in early layers! Use ReLU activation to mitigate.

What is the exploding gradient problem in deep neural networks?

- ❶ Gradients become very small during backpropagation.
- ❷ Gradients become very large during backpropagation, causing unstable updates.
- ❸ Gradients are zero for all parameters.
- ❹ Gradients oscillate in a regular pattern.

Answer: B **Explanation:** Exploding gradients: gradients become very large during backpropagation, leading to large parameter updates and unstable training. Loss may diverge.

Why Others Are Wrong:

- ❶ **A:** This describes vanishing gradients (small gradients).
- ❸ **C:** This is the extreme opposite - zero gradients.
- ❹ **D:** Regular oscillations are a different phenomenon.

Exam Trap: Exploding gradients = **very large gradients** causing instability! Use gradient clipping to mitigate.

What is backpropagation in neural networks?

- ❶ A method to initialize neural network weights.
- ❷ An algorithm that computes gradients of the loss function with respect to all weights using the chain rule.
- ❸ A method to choose the number of hidden layers.
- ❹ A method to select the activation function.

Answer: B Explanation: Backpropagation computes gradients of loss wrt all weights using chain rule. Propagates errors backward from output to input layers.

Why Others Are Wrong:

- ❶ **A:** This describes weight initialization - a separate step.
- ❸ **C:** This describes architecture selection - a different problem.
- ❹ **D:** This describes activation function selection - not backpropagation.

Exam Trap: Backpropagation uses the **chain rule** to compute gradients! This is the fundamental algorithm for neural network training.

Which mathematical rule is the foundation of backpropagation?

- 1 The product rule
- 2 The quotient rule
- 3 The chain rule
- 4 The power rule

Answer: C Explanation: Backpropagation is based on chain rule of calculus. Loss function is composition of many functions (layers, activations), chain rule propagates gradients backward.

Why Others Are Wrong:

- **A:** Product rule applies to products of functions - not the core of backprop.
- **B:** Quotient rule applies to ratios - not the core of backprop.
- **D:** Power rule applies to powers - not the core of backprop.

Exam Trap: Backpropagation = chain rule applied to neural networks! This is the single most important mathematical concept for neural network training.

What happens during the forward pass in backpropagation?

- ❶ Gradients are computed and weights are updated.
- ❷ Input data is passed through the network to compute the output and loss.
- ❸ The loss function is minimized.
- ❹ The weights are initialized.

Answer: B **Explanation:** Forward pass: input $\rightarrow$ hidden layers $\rightarrow$ output, compute output and loss. First step in backpropagation.

Why Others Are Wrong:

- ❶ **A:** This describes backward pass (gradient computation and update).
- ❸ **C:** Optimization happens over many iterations, not just the forward pass.
- ❹ **D:** Initialization happens before training begins.

Exam Trap: Forward pass computes **output and loss**; backward pass computes gradients! The order is critical.

What happens during the backward pass in backpropagation?

- 1 Input data is passed through the network.
- 2 Gradients of the loss with respect to each weight are computed using the chain rule.
- 3 The learning rate is adjusted.
- 4 The loss function is evaluated.

Answer: B Explanation: Backward pass computes gradients of loss wrt each weight using chain rule. Propagates errors from output layer back to input layer.

Why Others Are Wrong:

- **A:** This describes forward pass (input through network).
- **C:** Learning rate adjustment is a separate process (scheduling).
- **D:** Loss evaluation happens during forward pass.

Exam Trap: Backward pass computes **gradients** of all weights! The name "backward" comes from propagating errors backward from output to input.

After computing gradients in backpropagation, how are the weights updated?

- ① $w_{new} = w_{old} + \alpha \nabla L(w)$
- ② $w_{new} = w_{old} - \alpha \nabla L(w)$
- ③ $w_{new} = \nabla L(w)$
- ④ $w_{new} = w_{old} \times \alpha$

Answer: B **Explanation:** $w_{new} = w_{old} - \alpha \nabla L(w)$. Subtract gradient to move in direction of decreasing loss.

Why Others Are Wrong:

- **A:** This is gradient ascent - would increase loss (maximization).
- **C:** Ignores learning rate and old weights - not a valid update.
- **D:** Multiplication alone doesn't use gradient information.

Exam Trap: Weights updated by **subtracting** gradient $\times$ learning rate! This is the gradient descent update rule.

What is a computational graph in the context of backpropagation?

- ❶ A graph showing the architecture of the neural network.
- ❷ A graph representing the forward computation as a directed acyclic graph (DAG) that enables efficient gradient computation.
- ❸ A graph showing the training data distribution.
- ❹ A graph showing the loss function.

Answer: B Explanation: Computational graph represents forward computation as DAG of operations. Enables efficient gradient computation via automatic differentiation.

Why Others Are Wrong:

- ❶ **A:** This is a network architecture diagram - not the computational graph.
- ❸ **C:** This is data distribution - unrelated to computational graph.
- ❹ **D:** This is a loss function plot - not the computational graph.

Exam Trap: Computational graphs enable [automatic differentiation](#) and backpropagation! Foundation of deep learning frameworks like TensorFlow and PyTorch.

What is automatic differentiation in deep learning frameworks?

- ❶ Automatic selection of the learning rate.
- ❷ Automatic computation of gradients using the chain rule on computational graphs.
- ❸ Automatic selection of the number of layers.
- ❹ Automatic data preprocessing.

Answer: B Explanation: Automatic differentiation (autodiff) = automatic gradient computation using chain rule on computational graphs. Frameworks use autodiff to implement backpropagation.

Why Others Are Wrong:

- ❶ **A:** Learning rate selection is hyperparameter tuning - separate concept.
- ❸ **C:** Architecture selection is a separate problem (NAS).
- ❹ **D:** Data preprocessing happens before model training.

Exam Trap: Automatic differentiation is how frameworks implement **backpropagation**! This is the key technical insight.

What is a key memory requirement for backpropagation?

- ❶ Backpropagation requires storing all intermediate activations from the forward pass.
- ❷ Backpropagation requires no memory beyond the model parameters.
- ❸ Backpropagation requires storing only the final loss.
- ❹ Backpropagation requires storing only the gradients.

Answer: A Explanation: Backpropagation requires storing all intermediate activations (outputs of each layer) from forward pass. Needed during backward pass to compute gradients.

Why Others Are Wrong:

- ❷ **B:** False - memory is needed for intermediate activations.
- ❸ **C:** False - final loss alone is insufficient for gradient computation.
- ❹ **D:** False - gradients are computed during backward pass, not stored from forward pass.

Exam Trap: Backpropagation requires storing **all intermediate activations**! This is why deep networks are memory-intensive.

What is the correct order of operations in a training loop with backpropagation?

- ① Forward pass $\rightarrow$ Compute loss $\rightarrow$ Backward pass $\rightarrow$ Update weights
- ② Backward pass $\rightarrow$ Compute loss $\rightarrow$ Forward pass $\rightarrow$ Update weights
- ③ Compute loss $\rightarrow$ Forward pass $\rightarrow$ Backward pass $\rightarrow$ Update weights
- ④ Update weights $\rightarrow$ Forward pass $\rightarrow$ Compute loss $\rightarrow$ Backward pass

Answer: A Explanation: Correct order: (1) Forward pass: compute predictions and loss, (2) Compute loss, (3) Backward pass: compute gradients, (4) Update weights.

Why Others Are Wrong:

- **B:** Backward pass cannot happen before forward pass and loss computation.
- **C:** Loss cannot be computed before forward pass.
- **D:** Weights cannot be updated before computing gradients.

Exam Trap: Forward $\rightarrow$ Loss $\rightarrow$ Backward $\rightarrow$ Update! This order is absolutely critical and tested frequently.

Which of the following statements about backpropagation is FALSE?

- ① Backpropagation computes gradients of the loss with respect to all weights.
- ② Backpropagation uses the chain rule of calculus.
- ③ Backpropagation requires storing intermediate activations.
- ④ Backpropagation always finds the global minimum of the loss function.

Answer: D Explanation: Backpropagation does NOT guarantee finding the global minimum. Can get stuck in local minima or saddle points.

Why Others Are Wrong:

- **A:** True - this is exactly what backpropagation does.
- **B:** True - chain rule is the mathematical foundation.
- **C:** True - intermediate activations are stored for gradient computation.

Exam Trap: Backpropagation does NOT guarantee finding the global minimum! This is a common misconception.

What is the bias-variance tradeoff in supervised learning?

- ❶ The tradeoff between model complexity and interpretability.
- ❷ The tradeoff between the model's ability to fit the training data (bias) and its ability to generalize to new data (variance).
- ❸ The tradeoff between training speed and accuracy.
- ❹ The tradeoff between the number of features and the number of observations.

Answer: B Explanation: Bias-variance tradeoff: tension between fitting training data (low bias) and generalizing to new data (low variance). Simple models: high bias, low variance. Complex models: low bias, high variance.

Why Others Are Wrong:

- ❶ **A:** This is the accuracy-interpretability tradeoff - a different concept.
- ❸ **C:** This is the speed-accuracy tradeoff - not bias-variance.
- ❹ **D:** This describes the curse of dimensionality - a different concept.

Exam Trap: Bias = **error from assumptions**; Variance = **error from sensitivity to training data**! Understanding this distinction is critical.

What does "bias" refer to in the bias-variance tradeoff?

- ❶ The error introduced by the model's assumptions and limitations.
- ❷ The error introduced by using a limited training sample.
- ❸ The error introduced by noise in the data.
- ❹ The error introduced by the optimization algorithm.

Answer: A Explanation: Bias = error from model's assumptions and limitations. High bias = strong assumptions, underfitting (e.g., linear regression for nonlinear data).

Why Others Are Wrong:

- ❷ **B:** This describes variance (sensitivity to training sample).
- ❸ **C:** This describes irreducible error (noise in data).
- ❹ **D:** This describes optimization error - a different concept.

Exam Trap: Bias = **error from model assumptions** (underfitting)! Not from data noise or sampling.

What does "variance" refer to in the bias-variance tradeoff?

- ❶ The error introduced by the model's assumptions.
- ❷ The error introduced by the model's sensitivity to fluctuations in the training data.
- ❸ The error introduced by noise in the data.
- ❹ The error introduced by the choice of loss function.

Answer: B **Explanation:** Variance = error from model's sensitivity to training data fluctuations. High variance = overfitting (fits training data too closely, fails on new data).

Why Others Are Wrong:

- ❶ **A:** This describes bias (model assumptions).
- ❸ **C:** This describes irreducible error (data noise).
- ❹ **D:** This describes a different type of error.

Exam Trap: Variance = error from sensitivity to training data (overfitting)! Not from model assumptions or data noise.

Which of the following is a characteristic of a model with high bias?

- ❶ It fits the training data very well.
- ❷ It performs poorly on both training and test data (underfitting).
- ❸ It performs well on training data but poorly on test data.
- ❹ It has high variance.

Answer: B **Explanation:** High bias (underfitting): performs poorly on both training and test data. Model has made overly strong assumptions and failed to capture patterns.

Why Others Are Wrong:

- ❶ **A:** This describes low bias (fits training data well).
- ❸ **C:** This describes high variance (overfitting).
- ❹ **D:** High bias is associated with low variance, not high.

Exam Trap: High bias = **poor performance on both training and test** (underfitting)! Not just poor test performance.

Which of the following is a characteristic of a model with high variance?

- ❶ It performs poorly on both training and test data.
- ❷ It performs very well on training data but poorly on test data (overfitting).
- ❸ It has low complexity.
- ❹ It has high bias.

Answer: B **Explanation:** High variance (overfitting): performs very well on training data (memorizes patterns) but poorly on test data (fails to generalize).

Why Others Are Wrong:

- ❶ **A:** This describes high bias (underfitting).
- ❸ **C:** High variance is associated with high complexity, not low.
- ❹ **D:** High variance is associated with low bias, not high.

Exam Trap: High variance = **good training, poor test** (overfitting)! This is the classic signature of overfitting.

What is underfitting in supervised learning?

- ❶ When the model is too complex and learns noise in the data.
- ❷ When the model is too simple to capture the underlying patterns in the data.
- ❸ When the model performs well on training data but poorly on test data.
- ❹ When the model has too many parameters.

Answer: B **Explanation:** Underfitting: model too simple (e.g., linear for nonlinear data). Fails to capture underlying patterns. Poor performance on both training and test data.

Why Others Are Wrong:

- **A:** This describes overfitting (too complex, learns noise).
- **C:** This describes overfitting (good training, poor test).
- **D:** Too many parameters leads to overfitting, not underfitting.

Exam Trap: Underfitting = **model too simple** = poor training and test performance! Not just poor test performance.

What is overfitting in supervised learning?

- ❶ When the model is too simple to capture the underlying patterns.
- ❷ When the model is too complex and learns the noise in the training data.
- ❸ When the model performs poorly on both training and test data.
- ❹ When the model has too few parameters.

Answer: B Explanation: Overfitting: model too complex (e.g., high-degree polynomial). Learns noise in training data. High training accuracy but poor test accuracy.

Why Others Are Wrong:

- **A:** This describes underfitting (too simple).
- **C:** This describes underfitting (poor both).
- **D:** Too few parameters leads to underfitting, not overfitting.

Exam Trap: Overfitting = **model too complex** = good training, poor test! This is the classic signature.

Which of the following is a common cause of overfitting?

- ① Too few features.
- ② Too many parameters relative to the number of training samples.
- ③ Too much regularization.
- ④ Too simple a model.

Answer: B Explanation: Overfitting caused by too many parameters (or too complex model) relative to training samples. Model memorizes training data instead of learning general patterns.

Why Others Are Wrong:

- **A:** Too few features leads to underfitting, not overfitting.
- **C:** Too much regularization leads to underfitting.
- **D:** Too simple a model leads to underfitting.

Exam Trap: Overfitting happens when **too many parameters for too few samples!** This is the key cause.

Which of the following is a common cause of underfitting?

- ❶ Too many parameters.
- ❷ Insufficient model complexity (e.g., linear model for nonlinear data).
- ❸ Too little regularization.
- ❹ Too much training data.

Answer: B **Explanation:** Underfitting caused by insufficient model complexity. Example: linear model for nonlinear data. Model can't capture patterns.

Why Others Are Wrong:

- **A:** Too many parameters leads to overfitting.
- **C:** Too little regularization leads to overfitting.
- **D:** More training data generally helps, not causes underfitting.

Exam Trap: Underfitting happens when **model is too simple** for the data! This is the key cause.

As model complexity increases, what happens to bias and variance?

- ❶ Bias decreases and variance decreases.
- ❷ Bias decreases and variance increases.
- ❸ Bias increases and variance decreases.
- ❹ Bias increases and variance increases.

Answer: B Explanation: As complexity increases: bias decreases (captures more complex patterns), variance increases (more sensitive to training data). This is the tradeoff.

Why Others Are Wrong:

- ❶ **A:** Both decrease is incorrect - variance increases with complexity.
- ❸ **C:** This is the opposite direction (complexity increases $\rightarrow$ bias $\downarrow$, variance $\uparrow$).
- ❹ **D:** Both increase is incorrect - bias decreases with complexity.

Exam Trap: More complexity $\rightarrow$ **lower bias, higher variance!** This is the fundamental tradeoff direction.

Where is the optimal model complexity located in the bias-variance tradeoff?

- ❶ At the point of highest variance.
- ❷ At the point where total error ($\text{bias}^2 + \text{variance} + \text{irreducible error}$) is minimized.
- ❸ At the point of lowest bias.
- ❹ At the point of highest bias.

Answer: B Explanation: Optimal complexity at minimum total error: $\text{bias}^2 + \text{variance} + \text{irreducible error}$. Balances underfitting and overfitting.

Why Others Are Wrong:

- ❶ **A:** Highest variance = overfitting (too complex).
- ❸ **C:** Lowest bias = overfitting (too complex).
- ❹ **D:** Highest bias = underfitting (too simple).

Exam Trap: Optimal complexity = **minimum total error** ($\text{bias}^2 + \text{variance}$)! Not at extremes.

A simple linear regression model for a nonlinear relationship would likely have:

- ❶ High bias, low variance (underfitting).
- ❷ Low bias, high variance (overfitting).
- ❸ Low bias, low variance (ideal).
- ❹ High bias, high variance.

Answer: A **Explanation:** Linear model for nonlinear data is too simple. High bias (can't capture nonlinear patterns), low variance (not sensitive to training data). Underfitting.

Why Others Are Wrong:

- ❷ **B:** This describes complex model (e.g., high-degree polynomial).
- ❸ **C:** Ideal but unrealistic for mismatched model.
- ❹ **D:** Unlikely combination for linear model on nonlinear data.

Exam Trap: Too simple → **high bias, low variance** (underfitting)! This is the signature of underfitting.

A high-degree polynomial regression model on a small dataset would likely have:

- ❶ High bias, low variance (underfitting).
- ❷ Low bias, high variance (overfitting).
- ❸ Low bias, low variance (ideal).
- ❹ High bias, high variance.

Answer: B Explanation: High-degree polynomial on small dataset is too complex. Low bias (fits training data very well), high variance (very sensitive to training data). Overfitting.

Why Others Are Wrong:

- ❶ **A:** This describes simple model (underfitting).
- ❸ **C:** Ideal but unrealistic for complex model on small dataset.
- ❹ **D:** Unlikely combination for high-degree polynomial.

Exam Trap: Too complex → **low bias, high variance** (overfitting)! This is the signature of overfitting.

What is irreducible error in the bias-variance decomposition?

- ❶ Error that can be reduced by increasing model complexity.
- ❷ Error that can be reduced by using more training data.
- ❸ Error that is due to noise in the data and cannot be reduced by any model.
- ❹ Error that is due to the choice of loss function.

Answer: C Explanation: Irreducible error = noise in data (measurement errors, inherent randomness). Cannot be reduced by any model regardless of complexity.

Why Others Are Wrong:

- ❶ **A:** This describes reducible error (bias) - can be reduced.
- ❷ **B:** This describes reducible error (variance) - can be reduced.
- ❹ **D:** Loss function choice is a modeling decision, not irreducible error.

Exam Trap: Irreducible error = **noise in the data** - cannot be eliminated! Even the best possible model has irreducible error.

Which of the following statements about the bias-variance tradeoff is TRUE?

- ❶ Simple models tend to have low bias and high variance.
- ❷ Complex models tend to have high bias and low variance.
- ❸ The optimal model balances bias and variance to minimize total error.
- ❹ Bias and variance are independent of each other.

Answer: C Explanation: Optimal model balances bias and variance to minimize total error ($\text{bias}^2 + \text{variance} + \text{irreducible error}$).

Why Others Are Wrong:

- ❶ **A:** False - simple models have high bias, low variance.
- ❷ **B:** False - complex models have low bias, high variance.
- ❹ **D:** False - bias and variance are related through model complexity.

Exam Trap: Optimal model **balances bias and variance!** Not at extremes, and not independent.

What is regularization in supervised learning?

- ❶ A technique to increase model complexity.
- ❷ A technique to reduce overfitting by adding a penalty term to the loss function.
- ❸ A technique to increase the training data size.
- ❹ A technique to select the best features.

Answer: B Explanation: Regularization prevents overfitting by adding penalty term to loss function that discourages large parameter values or complex models.

Why Others Are Wrong:

- **A:** This is the opposite of regularization (which reduces complexity).
- **C:** This describes data augmentation - a different technique.
- **D:** This describes feature selection - related but distinct.

Exam Trap: Regularization adds a **penalty** to the loss function! Not increasing complexity or data.

Q62: L1 Regularization (Lasso)

What is L1 regularization (Lasso) and what is its key property?

- ❶ Adds $\lambda \sum w_i^2$ to the loss; shrinks weights toward zero.
- ❷ Adds $\lambda \sum |w_i|$ to the loss; can set weights to exactly zero (feature selection).
- ❸ Adds $\lambda \sum w_i$ to the loss; increases weights.
- ❹ Adds $\lambda \sum |w_i|^3$ to the loss; has no special property.

Answer: B Explanation: L1 adds $\lambda \sum |w_i|$ to loss. Key property: can set weights to exactly zero, performing feature selection. Also known as Lasso.

Why Others Are Wrong:

- ❶ **A:** This describes L2 regularization (Ridge) - shrinks but doesn't zero out.
- ❸ **C:** Linear penalty is not standard and would increase weights.
- ❹ **D:** Cubic penalty is not a standard regularization technique.

Exam Trap: L1 regularization **can set weights to zero** (feature selection)! This is the key differentiator from L2.

Q63: L2 Regularization (Ridge)

What is L2 regularization (Ridge) and what is its key property?

- ❶ Adds $\lambda \sum |w_i|$ to the loss; can set weights to zero.
- ❷ Adds $\lambda \sum w_i^2$ to the loss; shrinks weights toward zero but does not set them to zero.
- ❸ Adds $\lambda \sum w_i$ to the loss; increases weights.
- ❹ Adds $\lambda \sum |w_i|^3$ to the loss; has no special property.

Answer: B Explanation: L2 adds $\lambda \sum w_i^2$ to loss. Shrinks weights toward zero but doesn't set them to zero. Also known as Ridge regression.

Why Others Are Wrong:

- ❶ **A:** This describes L1 regularization (Lasso) - can set to zero.
- ❸ **C:** Linear penalty is not standard regularization.
- ❹ **D:** Cubic penalty is not standard regularization.

Exam Trap: L2 regularization **shrinks weights** but does not set them to zero! This is the key differentiator from L1.

What is Elastic Net regularization?

- ① A combination of L1 and L2 regularization: $\lambda_1 \sum |w_i| + \lambda_2 \sum w_i^2$
- ② A method that only uses L1 regularization.
- ③ A method that only uses L2 regularization.
- ④ A method that adds no regularization.

Answer: A Explanation: Elastic Net combines L1 and L2: $\lambda_1 \sum |w_i| + \lambda_2 \sum w_i^2$. Benefits from both feature selection (L1) and weight shrinkage (L2).

Why Others Are Wrong:

- **B:** This is Lasso (L1 only) - incomplete.
- **C:** This is Ridge (L2 only) - incomplete.
- **D:** This is no regularization - not Elastic Net.

Exam Trap: Elastic Net = **L1 + L2** regularization! It combines both penalties.

What happens as the regularization parameter λ increases?

- ❶ The model becomes more complex.
- ❷ The model becomes simpler (weights shrink more toward zero).
- ❸ The model becomes more prone to overfitting.
- ❹ The model has no change.

Answer: B **Explanation:** As λ increases, penalty on large weights becomes stronger, weights shrink more toward zero. Model becomes simpler (helps prevent overfitting).

Why Others Are Wrong:

- ❶ **A:** Larger λ means more regularization = simpler model, not more complex.
- ❸ **C:** Larger λ reduces overfitting, doesn't increase it.
- ❹ **D:** λ definitely changes the model - larger = simpler.

Exam Trap: Larger $\lambda \rightarrow$ **simpler model** (more regularization)! Not more complex.

How does regularization prevent overfitting?

- ❶ By increasing the number of features.
- ❷ By penalizing large parameter values, which reduces the model's complexity and sensitivity to training data.
- ❸ By increasing the training data size.
- ❹ By using a more complex loss function.

Answer: B Explanation: Regularization penalizes large parameter values. Reduces model complexity and sensitivity to training data fluctuations, improving generalization.

Why Others Are Wrong:

- ❶ **A:** Increasing features increases overfitting risk, doesn't prevent it.
- ❸ **C:** This describes data augmentation, not regularization.
- ❹ **D:** More complex loss function isn't regularization.

Exam Trap: Regularization = penalizes large weights to reduce overfitting! This is the mechanism.

Is dropout a form of regularization? If so, how does it work?

- ❶ No, dropout is not a form of regularization.
- ❷ Yes, dropout randomly drops neurons during training, creating an ensemble effect and preventing co-adaptation.
- ❸ Yes, dropout adds a penalty term to the loss function.
- ❹ No, dropout only affects the forward pass, not the backward pass.

Answer: B Explanation: Dropout is a regularization technique (and ensemble technique). Randomly drops (sets to zero) a fraction of neurons during each training iteration. Prevents co-adaptation.

Why Others Are Wrong:

- ❶ **A:** False - dropout IS a form of regularization.
- ❸ **C:** Dropout doesn't add a penalty term - it's a different mechanism.
- ❹ **D:** Dropout affects both forward and backward passes.

Exam Trap: Dropout is a **regularization technique** for neural networks! It works by randomly dropping neurons, not by adding penalty.

Q68: Early Stopping as Regularization

Is early stopping a form of regularization? If so, how does it work?

- ❶ No, early stopping is not a form of regularization.
- ❷ Yes, early stopping stops training before convergence, limiting the model's complexity and preventing overfitting.
- ❸ Yes, early stopping adds a penalty term to the loss function.
- ❹ No, early stopping only affects the training speed, not the model.

Answer: B Explanation: Early stopping is a regularization technique. Stops training when validation error starts to increase, limiting model complexity and preventing overfitting.

Why Others Are Wrong:

- ❶ **A:** False - early stopping IS a form of regularization.
- ❸ **C:** Early stopping doesn't add penalty term - different mechanism.
- ❹ **D:** Early stopping affects model (stops before overfitting).

Exam Trap: Early stopping is a **regularization technique** that stops training early! It's a form of "implicit regularization."

How does regularization affect the bias-variance tradeoff?

- ❶ Regularization increases both bias and variance.
- ❷ Regularization decreases bias and increases variance.
- ❸ Regularization increases bias and decreases variance.
- ❹ Regularization has no effect on bias or variance.

Answer: C Explanation: Regularization increases bias (simpler model may not fit as well) and decreases variance (less sensitive to training data). This is the tradeoff.

Why Others Are Wrong:

- ❶ **A:** Both increase is incorrect - variance decreases.
- ❷ **B:** This is the opposite direction (regularization $\rightarrow$ bias $\uparrow$, variance $\downarrow$).
- ❹ **D:** False - regularization definitely affects both.

Exam Trap: Regularization $\rightarrow$ **increases bias, decreases variance!** This is how it prevents overfitting.

When should regularization be applied in supervised learning?

- ❶ When the model is underfitting.
- ❷ When the model is overfitting (high variance).
- ❸ When the model has too few parameters.
- ❹ When the training data is perfectly clean.

Answer: B Explanation: Regularization should be applied when model is overfitting (high variance). It reduces overfitting by penalizing large weights and limiting complexity.

Why Others Are Wrong:

- ❶ **A:** Underfitting would be worsened by regularization (makes model simpler).
- ❸ **C:** Too few parameters = underfitting - regularization would make worse.
- ❹ **D:** Perfectly clean data doesn't need regularization (though still might help).

Exam Trap: Use regularization when **overfitting** (high variance)! Not when underfitting.

Which regularization method is more likely to perform feature selection?

- ❶ L2 regularization (Ridge)
- ❷ L1 regularization (Lasso)
- ❸ Both perform feature selection equally.
- ❹ Neither performs feature selection.

Answer: B Explanation: L1 (Lasso) performs feature selection because it can set weights to exactly zero. L2 (Ridge) shrinks weights but doesn't zero them out.

Why Others Are Wrong:

- ❶ **A:** L2 only shrinks weights, doesn't zero them - no feature selection.
- ❸ **C:** They are not equal - L1 does feature selection, L2 doesn't.
- ❹ **D:** L1 definitely performs feature selection.

Exam Trap: L1 (Lasso) = **feature selection**; L2 (Ridge) = **weight shrinkage**! This is a key distinction.

In the context of regularization, what does a weight of zero indicate?

- ❶ The feature is very important.
- ❷ The feature is not used by the model (feature selection).
- ❸ The feature has high variance.
- ❹ The feature has high bias.

Answer: B Explanation: Weight of zero indicates feature is not used by model. This is feature selection (especially from L1 regularization).

Why Others Are Wrong:

- ❶ **A:** Weight of zero means feature has no importance.
- ❸ **C:** Weight value doesn't directly indicate variance.
- ❹ **D:** Weight value doesn't directly indicate bias.

Exam Trap: Weight = 0 $\rightarrow$ **feature not used** (feature selection)! This is the interpretation.

If λ is too large in regularization, what happens?

- ❶ The model overfits.
- ❷ The model underfits.
- ❸ The model performs optimally.
- ❹ The model has no effect.

Answer: B **Explanation:** If λ too large, penalty too strong, all weights shrink to zero. Model becomes too simple and underfits.

Why Others Are Wrong:

- ❶ **A:** Too small λ leads to overfitting, not too large.
- ❷ **C:** Optimal requires balanced λ - not too large or small.
- ❹ **D:** λ definitely affects model - too large causes underfitting.

Exam Trap: Too large $\lambda \rightarrow$ **underfitting!** Too small $\lambda \rightarrow$ overfitting. Need balance.

Which of the following is a form of regularization?

- ① L1 regularization (Lasso)
- ② L2 regularization (Ridge)
- ③ Dropout
- ④ All of the above

Answer: D **Explanation:** L1, L2, and dropout are all forms of regularization. They prevent overfitting through different mechanisms.

Why Others Are Wrong:

- **A:** Correct but incomplete - other options also valid.
- **B:** Correct but incomplete - other options also valid.
- **C:** Correct but incomplete - other options also valid.

Exam Trap: Many techniques are forms of regularization! Don't assume only L1/L2 - dropout and early stopping also count.

What is the purpose of the regularization parameter λ ?

- ❶ To control the tradeoff between fitting the training data and keeping the model simple.
- ❷ To control the learning rate.
- ❸ To control the number of epochs.
- ❹ To control the batch size.

Answer: A Explanation: λ controls tradeoff between fitting training data (small λ) and keeping model simple (large λ).

Why Others Are Wrong:

- ❷ **B:** Learning rate is α - different hyperparameter.
- ❸ **C:** Number of epochs is a different hyperparameter.
- ❹ **D:** Batch size is a different hyperparameter.

Exam Trap: λ controls the **bias-variance tradeoff** in regularization! Not learning rate, epochs, or batch size.

What is cross-validation in supervised learning?

- ❶ A method to increase the training data size.
- ❷ A resampling technique used to evaluate model performance and select hyperparameters.
- ❸ A method to reduce overfitting by adding a penalty term.
- ❹ A method to initialize model parameters.

Answer: B Explanation: Cross-validation partitions data into training and validation sets multiple times to get reliable performance estimates and select hyperparameters.

Why Others Are Wrong:

- ❶ **A:** This describes data augmentation - different technique.
- ❸ **C:** This describes regularization - different technique.
- ❹ **D:** This describes weight initialization - different concept.

Exam Trap: Cross-validation evaluates **model performance** and selects **hyperparameters**! Not data augmentation or regularization.

How does k-fold cross-validation work?

- ❶ Split data into k parts, train on one part and validate on the remaining k-1 parts.
- ❷ Split data into k parts, train on k-1 parts and validate on the remaining 1 part, repeat k times.
- ❸ Split data into k parts and use all parts for training.
- ❹ Split data into k parts and use all parts for validation.

Answer: B **Explanation:** Split into k folds. For each iteration: train on k-1 folds, validate on 1 fold. Repeat k times. Average results.

Why Others Are Wrong:

- ❶ **A:** Reverses training and validation (would train on too little data).
- ❸ **C:** Using all data for training = no validation - not cross-validation.
- ❹ **D:** Using all data for validation = no training - not cross-validation.

Exam Trap: K-fold CV = train on **k-1 folds**, validate on **1 fold**, repeat k times! This is the exact procedure.

What is stratified cross-validation?

- 1 Cross-validation where each fold has the same number of observations.
- 2 Cross-validation where each fold preserves the same class distribution as the full dataset.
- 3 Cross-validation where the folds are chosen randomly.
- 4 Cross-validation where the folds are chosen sequentially.

Answer: B Explanation: Stratified CV ensures each fold has same class distribution (proportion of each class) as the full dataset. Important for imbalanced datasets.

Why Others Are Wrong:

- A: Equal fold sizes is standard k-fold, not stratification.
- C: Random folds may not preserve class distribution.
- D: Sequential folds may introduce order bias.

Exam Trap: Stratified CV preserves **class proportions** in each fold! This is the key differentiator.

When is stratified cross-validation particularly important?

- ❶ When the dataset is perfectly balanced.
- ❷ When the dataset is imbalanced (one class is much rarer than others).
- ❸ When the dataset is very large.
- ❹ When the dataset has no categorical variables.

Answer: B Explanation: Stratified CV is especially important for imbalanced datasets. Without it, some folds might have very few samples from the rare class.

Why Others Are Wrong:

- ❶ **A:** Balanced datasets don't need stratification (though it doesn't hurt).
- ❸ **C:** Dataset size doesn't determine need for stratification.
- ❹ **D:** Categorical variables not relevant - stratification is about class distribution.

Exam Trap: Stratified CV is essential for **imbalanced datasets**! This is the primary use case.

What is Leave-One-Out Cross-Validation (LOOCV)?

- ❶ A special case of k-fold CV where $k = N$ (one observation per fold).
- ❷ A special case of k-fold CV where $k = 2$.
- ❸ A special case of k-fold CV where $k = 10$.
- ❹ A method that uses no validation set.

Answer: A **Explanation:** LOOCV = k-fold CV with $k = N$ (number of observations). Each fold has one observation. Train on $N-1$, validate on 1. Repeat N times.

Why Others Are Wrong:

- ❶ **B:** $k=2$ is 2-fold CV - not LOOCV (unless $N=2$).
- ❷ **C:** $k=10$ is 10-fold CV - most common but not LOOCV.
- ❸ **D:** LOOCV uses validation sets (one observation at a time).

Exam Trap: LOOCV = k-fold CV with $k = N$! This is the precise definition.

What is an advantage of Leave-One-Out Cross-Validation?

- ① It is computationally cheap.
- ② It uses almost all data for training each time ($N-1$ observations), giving low bias.
- ③ It always has low variance.
- ④ It works well for very large datasets.

Answer: B **Explanation:** LOOCV uses $N-1$ observations for training each time. This gives low bias (almost full dataset used for training).

Why Others Are Wrong:

- **A:** LOOCV is computationally expensive (requires N training runs).
- **C:** LOOCV can have high variance (estimates are correlated).
- **D:** LOOCV is not suitable for large datasets (too expensive).

Exam Trap: LOOCV has **low bias** but is **computationally expensive**! This is the key tradeoff.

What is a disadvantage of Leave-One-Out Cross-Validation?

- ❶ It has high bias.
- ❷ It is computationally expensive (requires N training runs).
- ❸ It always overfits.
- ❹ It cannot be used for classification.

Answer: B Explanation: LOOCV requires training model N times (once for each held-out observation). This is impractical for large datasets.

Why Others Are Wrong:

- **A:** LOOCV has low bias (uses $N-1$ training samples).
- **C:** LOOCV doesn't necessarily overfit (though variance can be high).
- **D:** LOOCV can be used for classification too.

Exam Trap: LOOCV is **computationally expensive** for large N ! This is the main disadvantage.

How does the choice of k in k -fold cross-validation affect the bias-variance tradeoff?

- ① Larger k increases bias and decreases variance.
- ② Larger k decreases bias and increases variance.
- ③ Larger k has no effect on bias or variance.
- ④ Smaller k decreases bias and increases variance.

Answer: B Explanation: Larger k = more training data ($k-1$ folds) $\rightarrow$ lower bias, higher variance. LOOCV ($k=N$) has lowest bias, highest variance.

Why Others Are Wrong:

- **A:** This is opposite - larger k decreases bias.
- **C:** False - k definitely affects bias-variance.
- **D:** Smaller k increases bias, decreases variance.

Exam Trap: Larger $k \rightarrow$ **lower bias, higher variance!** This is the tradeoff.

What is the most common value of k used in k-fold cross-validation?

- 1 $k = 2$
- 2 $k = 5$
- 3 $k = 10$
- 4 $k = N$ (LOOCV)

Answer: C **Explanation:** $k = 10$ is most common. Provides good balance between bias and variance and is computationally feasible.

Why Others Are Wrong:

- **A:** $k=2$ is sometimes used but not most common.
- **B:** $k=5$ is also common but $k=10$ is most common.
- **D:** LOOCV is used for small datasets, not most common.

Exam Trap: $k = 10$ is the **most common** value! This is a standard convention.

What is the primary purpose of cross-validation?

- ❶ To increase the training data size.
- ❷ To evaluate how well a model will generalize to new, unseen data.
- ❸ To reduce the number of features.
- ❹ To increase the learning rate.

Answer: B Explanation: Primary purpose: evaluate model generalization performance. Provides reliable estimate of test performance.

Why Others Are Wrong:

- ❶ **A:** Data augmentation increases data size - not CV.
- ❸ **C:** Feature selection reduces features - not CV.
- ❹ **D:** Learning rate is a training hyperparameter - not CV.

Exam Trap: Cross-validation estimates **generalization performance**! This is the fundamental purpose.

What is bootstrapping in the context of model evaluation?

- ① A resampling technique that samples with replacement from the dataset.
- ② A resampling technique that samples without replacement.
- ③ A method to add regularization.
- ④ A method to reduce the number of features.

Answer: A Explanation: Bootstrapping samples with replacement from dataset. Creates multiple bootstrap samples (same size as original) with some observations repeated and some omitted.

Why Others Are Wrong:

- **B:** Sampling without replacement is subsampling - not bootstrapping.
- **C:** Bootstrapping is not regularization.
- **D:** Bootstrapping is not feature selection.

Exam Trap: Bootstrapping = **sampling with replacement!** This is the key defining feature.

What is the primary purpose of bootstrapping in model evaluation?

- ❶ To estimate the variability of model performance (e.g., confidence intervals).
- ❷ To reduce overfitting.
- ❸ To increase the learning rate.
- ❹ To select the best features.

Answer: A Explanation: Bootstrapping estimates performance variability and confidence intervals. Trains multiple models on different bootstrap samples to get performance distribution.

Why Others Are Wrong:

- **B:** Bootstrapping is not regularization (though related to ensemble methods).
- **C:** Learning rate is a training parameter - not bootstrapping.
- **D:** Feature selection is a different technique.

Exam Trap: Bootstrapping estimates **performance variability**! This is the primary use.

Q88: Cross-Validation vs Bootstrapping

What is the key difference between cross-validation and bootstrapping?

- ❶ Cross-validation samples without replacement; bootstrapping samples with replacement.
- ❷ Cross-validation uses all data for training; bootstrapping uses only part of the data.
- ❸ Cross-validation is for classification; bootstrapping is for regression.
- ❹ Cross-validation and bootstrapping are identical.

Answer: A Explanation: CV (especially k-fold) samples without replacement (each observation appears in exactly one validation fold). Bootstrapping samples with replacement.

Why Others Are Wrong:

- ❶ **B:** False - both use parts of data for training/validation.
- ❷ **C:** False - both can be used for either task.
- ❸ **D:** False - they are different techniques.

Exam Trap: CV = without replacement; Bootstrapping = with replacement! This is the key difference.

What is grid search in the context of hyperparameter tuning?

- ❶ A method that randomly searches for optimal hyperparameters.
- ❷ A method that exhaustively searches over a predefined grid of hyperparameter values.
- ❸ A method that uses gradient descent to find hyperparameters.
- ❹ A method that uses cross-validation to select the best model.

Answer: B Explanation: Grid search exhaustively evaluates all combinations of hyperparameters from a predefined grid. Guarantees finding best within the grid but computationally expensive.

Why Others Are Wrong:

- ❶ **A:** This describes random search - different method.
- ❸ **C:** This describes gradient-based hyperparameter optimization.
- ❹ **D:** Cross-validation is used WITH grid search, not the same as grid search.

Exam Trap: Grid search = **exhaustive search** over a parameter grid! This is the defining feature.

What is a major disadvantage of grid search for hyperparameter tuning?

- ❶ It is computationally expensive for large grids.
- ❷ It always finds suboptimal hyperparameters.
- ❸ It cannot be used with cross-validation.
- ❹ It requires gradient information.

Answer: A Explanation: Grid search is computationally expensive, especially for large grids. Number of combinations grows exponentially with number of hyperparameters.

Why Others Are Wrong:

- **B:** False - it finds best within the grid (not suboptimal).
- **C:** False - grid search is often used with cross-validation.
- **D:** False - grid search doesn't require gradients.

Exam Trap: Grid search is **computationally expensive** for large grids! This is the main limitation.

What is the accuracy-interpretability tradeoff in supervised learning?

- ❶ The tradeoff between model complexity and training speed.
- ❷ The tradeoff between a model's predictive performance and how easily its decisions can be understood.
- ❸ The tradeoff between bias and variance.
- ❹ The tradeoff between the number of features and the number of observations.

Answer: B Explanation: Accuracy-interpretability tradeoff: tension between predictive performance and understandability. More complex models often more accurate but less interpretable.

Why Others Are Wrong:

- ❶ **A:** This is speed-complexity tradeoff - different concept.
- ❸ **C:** This is bias-variance tradeoff - different concept.
- ❹ **D:** This is curse of dimensionality - different concept.

Exam Trap: Accuracy vs Interpretability = **performance vs understandability**! This is the key concept.

Which of the following models typically has high interpretability?

- ① Deep neural networks
- ② Random forests
- ③ Decision trees (shallow)
- ④ Gradient boosting machines

Answer: C Explanation: Shallow decision trees have high interpretability. Decision-making process is easy to visualize and understand. "White-box" model.

Why Others Are Wrong:

- **A:** DNNs are black-box models (low interpretability).
- **B:** Random forests are ensembles of trees (less interpretable).
- **D:** GBM is complex ensemble (low interpretability).

Exam Trap: **Decision trees** are highly interpretable "white-box" models! This is a key example.

Which of the following models typically has high accuracy but low interpretability?

- ① Linear regression
- ② Decision trees (shallow)
- ③ Deep neural networks
- ④ Logistic regression

Answer: C Explanation: Deep neural networks achieve high accuracy (especially on complex tasks) but have low interpretability. Often called "black-box" models.

Why Others Are Wrong:

- **A:** Linear regression is highly interpretable.
- **B:** Shallow decision trees are highly interpretable.
- **D:** Logistic regression is highly interpretable.

Exam Trap: Deep neural networks = **high accuracy, low interpretability!** This is the classic example.

In which domain is interpretability particularly important?

- ❶ Image recognition
- ❷ Natural language processing
- ❸ Risk management and regulated industries (e.g., credit scoring)
- ❹ Recommendation systems

Answer: C Explanation: Interpretability is critical in risk management and regulated industries (credit scoring, healthcare, insurance) where decisions must be explainable to regulators and customers.

Why Others Are Wrong:

- ❶ **A:** Image recognition prioritizes accuracy over interpretability.
- ❷ **B:** NLP often prioritizes accuracy over interpretability.
- ❹ **D:** Recommendation systems prioritize accuracy and personalization.

Exam Trap: Interpretability is critical in **risk management and regulated industries!** This is the key domain.

In risk management, which type of model is often preferred despite potentially lower accuracy?

- ❶ Black-box models (e.g., deep neural networks)
- ❷ White-box models (e.g., decision trees, logistic regression)
- ❸ Ensemble models (e.g., random forests)
- ❹ Any model with the highest accuracy

Answer: B **Explanation:** Risk management prefers white-box (interpretable) models like decision trees and logistic regression. Regulatory requirements demand transparency.

Why Others Are Wrong:

- ❶ **A:** Black-box models are avoided despite higher accuracy.
- ❸ **C:** Ensembles are less interpretable than individual trees.
- ❹ **D:** Highest accuracy ignores interpretability concerns.

Exam Trap: Risk management prefers **interpretable (white-box)** models! This is the regulatory requirement.

Which of the following statements about the accuracy-interpretability tradeoff is TRUE?

- ❶ More interpretable models always have higher accuracy.
- ❷ Less interpretable models always have lower accuracy.
- ❸ There is often a tradeoff between accuracy and interpretability.
- ❹ Accuracy and interpretability are independent of each other.

Answer: C Explanation: There is often a tradeoff. Simpler, more interpretable models may have lower accuracy; complex, less interpretable models may have higher accuracy.

Why Others Are Wrong:

- ❶ **A:** False - simpler models often have lower accuracy.
- ❷ **B:** False - complex models often have higher accuracy.
- ❹ **D:** False - they are related through model complexity.

Exam Trap: Accuracy and interpretability are often **in tension** (tradeoff)! This is the key concept.

What is feature importance in the context of model interpretability?

- ❶ A measure of how important each feature is in making predictions.
- ❷ A measure of how much data is available for each feature.
- ❸ A measure of the variance of each feature.
- ❹ A measure of the correlation between features.

Answer: A Explanation: Feature importance measures how important each feature is in making predictions. Helps interpret models by identifying which features drive predictions.

Why Others Are Wrong:

- ❷ **B:** This is data availability - different concept.
- ❸ **C:** This is feature variance - different concept.
- ❹ **D:** This is correlation - different concept.

Exam Trap: Feature importance helps [interpret](#) model predictions! This is the key purpose.

What are **SHAP (SHapley Additive exPlanations)** values?

- ❶ A method to compute feature importance using game theory.
- ❷ A method to increase model accuracy.
- ❸ A method to regularize models.
- ❹ A method to select the best hyperparameters.

Answer: A **Explanation:** SHAP values use Shapley values from game theory. Provide unified framework for interpreting model predictions by attributing prediction to each feature.

Why Others Are Wrong:

- **B:** SHAP doesn't increase accuracy - it's for explanation.
- **C:** SHAP is not regularization.
- **D:** SHAP is not hyperparameter tuning.

Exam Trap: SHAP uses **game theory** to explain predictions! This is the key insight.

What is LIME (Local Interpretable Model-agnostic Explanations)?

- ❶ A method to explain individual predictions by approximating the model locally with an interpretable model.
- ❷ A method to increase model accuracy.
- ❸ A method to regularize models.
- ❹ A method to select the best features.

Answer: A **Explanation:** LIME explains individual predictions by approximating the complex model locally (near a specific prediction) with an interpretable model (e.g., linear regression).

Why Others Are Wrong:

- **B:** LIME doesn't increase accuracy - it's for explanation.
- **C:** LIME is not regularization.
- **D:** LIME is not feature selection (though it identifies important features locally).

Exam Trap: LIME provides **local explanations** for individual predictions! This is the key distinction from SHAP.

Which of the following provides the most comprehensive summary of supervised learning model estimation?

- ❶ Only OLS is used for model estimation.
- ❷ Model estimation involves choosing between OLS, NLS, and MLE, optimizing using gradient descent, and evaluating with cross-validation.
- ❸ Only MLE is used for model estimation.
- ❹ Model estimation is only about regularization.

Answer: B Explanation: Comprehensive estimation includes: choosing estimation method (OLS, NLS, MLE), optimizing (gradient descent), and evaluating (cross-validation).

Why Others Are Wrong:

- **A:** Only OLS is incomplete - NLS and MLE also used.
- **C:** Only MLE is incomplete - OLS also used.
- **D:** Regularization is part of model fitting, not the entire estimation process.

Exam Trap: Model estimation is **comprehensive** - includes method choice, optimization, and evaluation! This is the big picture.

Summary: Key Concepts

Core Estimation Methods:

- **OLS**: Minimizes SSE; closed-form for linear regression; BLUE under Gauss-Markov
- **MLE**: Maximizes likelihood; equivalent to OLS under normal errors
- **NLS**: For nonlinear-in-parameters models; requires numerical optimization

Optimization:

- **Gradient Descent**: Iterative; uses negative gradient; learning rate controls step size
- **Backpropagation**: Chain rule for neural networks; requires storing activations

Model Selection:

- **Bias-Variance Tradeoff**: Simplicity vs complexity
- **Regularization**: L1 (Lasso - feature selection), L2 (Ridge - shrinkage)
- **Cross-Validation**: k-fold (k=10 common), evaluates generalization

Exam Success: Understand the tradeoffs and know the definitions!